

NAIL094 — Decision Procedures and SAT/SMT Solvers

Watched Literals

Zerina Ahmetovic

1 Task overview

Lazy watched literals data structure is added to the DPLL solver of the preceding task [1]. Adjacency lists are to be used as an alternative and check on the benchmark examples how much does watched literals save on the number of checked clauses and running time.

Non-random benchmarks are to be used since random instances are not good for comparison with adjacency lists (they do not have long clauses). Therefore, the two non-random families from SATLIB [3] are recommended for this task, pigeonhole [5] and All Interval Series [4], collected in Table 1.

2 Implementation

As Watched Literals is an additon to the DPLL solver [1], both of them use the same four-method interface and are chosen by a template parameter of the solver, so the search loop, the trail and the decision heuristic are the same code in both runs. Only propagation differs. The solver is invoked as `dp11 [input]` for adjacency lists and `dp11 -watched [input]` for watched literals.

Adjacency lists. For each literal l , `occ[l]` lists the clauses containing it. When l becomes true every clause holding $\neg l$ has lost a literal, so each of them is visited and rescanned from the start to see whether it is satisfied, still open, unit, or falsified.

Watched literals. Every clause of length at least two watches two of its literals, and `watchList[l]` holds the clauses currently watching l . A clause is inspected only when one of its two watched literals becomes false; the engine then looks for any other non-false literal to move the watch to, and only if none exists is the clause unit or falsified. Unit and empty clauses cannot carry two watches and are handled once at initialisation.

The property that matters is what happens on backtracking: watches are not restored, because a watch on a literal that has just become unassigned is still a valid watch. Nothing is undone.

Counting. Both engines increment the same counter at the same point, once per clause taken off the list being walked and immediately before the clause is examined. This counts clause visits rather than distinct clauses, and it is the only reading under which the two numbers are comparable.

3 Experimental evaluation

The times in Table 1 were measured with g++ 14.2.0 (MSYS2 UCRT64) at `-std=c++20 -O2`. Each instance was run three times and the fastest kept, the timer covers the search only. On

every instance the two engines report identical decision and conflict counts. They therefore walk the same search tree, and the differences below are propagation and nothing else.

instance	vars	clauses	$k > 2$	result	clauses checked			time (s)	
					adjacency	watched	ratio	adjacency	watched
hole6	42	133	0.05	UNSAT	63 161	61 730	1.02	0.00089	0.00128
hole7	56	204	0.04	UNSAT	722 098	707 121	1.02	0.00939	0.01300
hole8	72	297	0.03	UNSAT	9 305 155	9 126 435	1.02	0.12262	0.15621
hole9	90	415	0.02	UNSAT	133 426 496	131 029 082	1.02	1.67565	2.21553
ais6	61	441	0.28	SAT	2 797	1 855	1.51	0.00014	0.00012
ais8	113	1149	0.27	SAT	51 553	31 531	1.64	0.00102	0.00084
ais10	181	2377	0.26	SAT	1 112 800	653 138	1.70	0.01909	0.01356
ais12	265	4269	0.26	SAT	30 580 645	17 250 426	1.77	0.37316	0.33094

Table 1: Adjacency lists against lazy watched literals on PHOLE and All Interval Series. $k > 2$ is the fraction of clauses with more than two literals. Times are the fastest of three runs and cover search only. Both engines produce identical decision and conflict counts on every instance, so the two columns differ in propagation alone.

Table 1 splits cleanly by family. On the pigeonhole instances watched literals check only 2% fewer clauses, a ratio of 1.02 on all four, and take 1.27 to 1.45 times as long to run. On All Interval Series they check 34 to 44% fewer, a ratio rising from 1.51 on `ais6` to 1.77 on `ais12`, and save 11 to 29% of the running time.

The $k > 2$ column of Table 1 accounts for the difference. A watch only ever moves inside its own clause, so what the structure saves is clause visits, and that is the quantity reported: adjacency lists enter a clause whenever any of its k literals is falsified, watched literals only when one of the two being watched is. A binary clause has no third literal for a watch to move to, and the pigeonhole instances are 95 to 98% binary: there is nothing to save, and maintaining the watch lists is left as pure overhead. All Interval Series carries 26 to 28% clauses longer than two literals, which is enough for the lazy invariant to pay for itself. The same reading explains the trend within each family — the ratio grows with instance size on All Interval Series, while on pigeonhole it stays at 1.02 throughout.

Code

The full source is in the accompanying repository [2]. `src/watched_literals.hpp` holds the watched engine and is the only file added for this task; it includes `src/dp11.hpp` and nothing in `dp11.hpp` refers back to it. The adjacency engine, the trail and the search loop are unchanged in `src/dp11.hpp`. `tests/check_watched.py` asserts that the two engines agree on every instance in the test suite and that their decision and conflict counts match, and `tests/bench_watched.py` produces the measurements.

Summary

Watched literals reduce the number of clauses checked exactly where clauses are long enough for a watch to have somewhere to move, and on those instances the reduction carries through to the running time. On the pigeonhole family, which is almost entirely binary clauses, they check barely fewer clauses and lose on time to their own bookkeeping, as Table 1 shows. What decides the outcome is the clause width of the instance rather than its size or its hardness.

References

- [1] Z. Ahmetovic. DPLL experimental evaluation, the preceding report of this series. Included in the accompanying repository.
- [2] Z. Ahmetovic. Source repository for the NAIL094 programming tasks. <https://github.com/inferno73/SAT-solver-NAIL094>
- [3] H. H. Hoos and T. Stützle. SATLIB: An Online Resource for Research on SAT. <https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html>
- [4] SATLIB benchmark problems, All Interval Series. <https://www.cs.ubc.ca/~hoos/SATLIB/Benchmarks/SAT/AIS/descr.html>
- [5] SATLIB benchmark problems, DIMACS pigeon hole. <https://www.cs.ubc.ca/~hoos/SATLIB/Benchmarks/SAT/DIMACS/PHOLE/descr.html>